

**LAPORAN PRAKTIKUM
MODUL 8
“ Queue”**



**Disusun Oleh:
Shilfi Habibah - 2311104002
S1SE07-01**

**Dosen :
Yudha Islami Sulistya**

**PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
INSTITUT TEKNOLOGI TELKOM PURWOKERTO
2024**

MODUL VIII

QUEUE

A. TUJUAN PRAKTIKUM

1. Mahasiswa mampu menjelaskan definisi dan konsep dari queue
2. Mahasiswa mampu menerapkan operasi tambah, menghapus pada queue
3. Mahasiswa mampu menerapkan operasi tampil data pada queue

B. DASAR TEORI

Queue adalah struktur data yang digunakan untuk menyimpan data dengan metode **FIFO** (First-In First-Out). Data yang pertama dimasukkan ke dalam queue akan menjadi data yang pertama pula untuk dikeluarkan dari queue. Queue mirip dengan konsep **antrian** pada kehidupan sehari-hari, dimana konsumen yang datang lebih dulu akan dilayani terlebih dahulu.

Implementasi queue dapat dilakukan dengan menggunakan array atau linked list. Struktur data queue terdiri dari dua pointer yaitu front dan rear. **Front/head** adalah pointer ke elemen pertama dalam queue dan **rear/tail/back** adalah pointer ke elemen terakhir dalam queue.



FIRST IN FIRST OUT (FIFO)

Perbedaan antara *stack* dan *queue* terletak pada aturan penambahan dan penghapusan elemen. Pada *stack*, operasi penambahan dan penghapusan elemen dilakukan di satu ujung yang disebut *top* (ujung atas). Elemen yang terakhir kali dimasukkan ke dalam *stack* akan berada di posisi paling atas dan akan menjadi elemen pertama yang dihapus. Sifat ini dikenal dengan istilah *LIFO* (Last In, First Out). Contoh analogi sederhana dari *stack*

adalah tumpukan piring, di mana piring terakhir yang ditambahkan berada di posisi paling atas dan akan diambil atau dihapus terlebih dahulu.

Sebaliknya, pada *queue*, operasi penambahan dan penghapusan elemen dilakukan di dua ujung yang berbeda. Elemen baru ditambahkan di ujung belakang (*rear* atau *tail*), dan elemen dihapus dari ujung depan (*front* atau *head*). Proses ini mengikuti prinsip *FIFO* (First In, First Out), yang berarti elemen pertama yang dimasukkan ke dalam *queue* akan menjadi elemen pertama yang dikeluarkan. Dalam konteks *queue*, operasi penambahan elemen dikenal sebagai **Enqueue**, dan operasi penghapusan elemen disebut **Dequeue**.

Pada *Enqueue*, elemen ditambahkan di belakang *queue* setelah elemen terakhir yang ada, sementara pada *Dequeue*, elemen paling depan (*head*) dihapus, dan posisi *head* akan bergeser ke elemen berikutnya. Contoh penggunaan *queue* dalam kehidupan sehari-hari adalah antrean di kasir, di mana orang pertama yang datang adalah yang pertama dilayani.

Operasi pada Queue

- `enqueue()` : menambahkan data ke dalam queue.
- `dequeue()` : mengeluarkan data dari queue.
- `peek()` : mengambil data dari queue tanpa menghapusnya.
- `isEmpty()` : mengecek apakah queue kosong atau tidak.
- `isFull()` : mengecek apakah queue penuh atau tidak.
- `size()` : menghitung jumlah elemen dalam queue.

C. GUIDED

Code:

```
#include <iostream>

using namespace std;

const int maksimalQueue = 5; // Maksimal antrian
int front = 0;               // Penanda antrian
int back = 0;                // Penanda
string queueTeller[5];      // Fungsi pengecekan

bool isFull()
{ // pengecekan antrian penuh atau tidak
    if (back == maksimalQueue) {
        return true; // =1
    } else {
        return false;
    }
}

bool isEmpty() { // Antriannya kosong atau tidak
    if (back == 0) {
        return true;
    } else {
        return false;
    }
}

void enqueueAntrian(string data) { // Fungsi menambahkan antrian
    if (isFull()) {
        cout << "Antrian penuh" << endl;
    } else { // Antriannya ada isi
        if (isEmpty()) { // kondisi ketika queue kosong
            queueTeller[0] = data;
            front++;
            back++;
        } else {
            queueTeller[back] = data;
            back++;
        }
    }
}
```

```

void dequeueAntrian() {
    if (isEmpty()){
        cout << "Antrian kosong" << endl;
    } else {
        for (int i = 0; i < back - 1; i++){
            queueTeller[i] = queueTeller[i + 1];
        }
        queueTeller[back - 1] = "";
        back--;
    }
}

int countQueue() { //Fungsi menghitung banyak antrian
    return back;
}

void clearQueue() { //Fungsi menghapus semua antrian
    if (isEmpty()){
        cout << "Antrian kosong" << endl;
    } else {
        for (int i = 0; i < back; i++) {
            queueTeller[i] = "";
        }
        back = 0;
        front = 0;
    }
}

void viewQueue() { // Fungsi melihat antrian
    cout << "Data antrian teller:" << endl;
    for (int i = 0; i < maksimalQueue; i++) {
        if (queueTeller[i] != "") {
            cout << i + 1 << ". " << queueTeller[i] << endl;
        } else {
            cout << i + 1 << ". (kosong)" << endl;
        }
    }
}

int main()
{
    enqueueAntrian("Shilfi");
    enqueueAntrian("Kelvin");
    viewQueue();
    cout << "Jumlah antrian = " << countQueue << endl;
    dequeueAntrian();
    viewQueue();
    cout << "Jumlah antrian = " << countQueue() << endl;
    clearQueue();
    viewQueue();
    cout << "Jumlah antrian = " << countQueue() << endl;
    return 0;
}

```

- Konstanta & Variabel : **const int maksimalQueue = 5;**(mendefinisikan ukuran antrian maksimum sebagai 5), **int front** dan **int back** (melacak posisi depan dan akhir antrian) dan **string queueTeller[5];**(array untuk menyimpan item antrian maksimum 5).
- Fungsi : **isFull()** (memeriksa apakah antrean sudah penuh), **isEmpty()** (memeriksa apakah antrean kosong), **enqueueAntrian(string data)** (menambahkan item ke antrean), **dequeueAntrian()** (menghapus item depan dari antrean), **countQueue()** (mengembalikan jumlah item dalam antrian), **clearQueue()** (menghapus antrean

dengan mengatur ulang semua item dan menyetel front dan back ke 0), **viewQueue()** (menampilkan isi antrian. Jika ada item, akan dicetak; jika tidak, akan dicetak "kosong")

Output:

Program menerapkan operasi dasar antrian dasar: enqueue, dequeue, count, view dan clear, serta menangani kasus-kasus khusus antrian penuh atau kosong. Output menampilkan yaitu pertama menambahkan “Shilfi” dan “Kelvin” pada antrian, lalu memanggil **viewQueue()** agar menampilkan isi antrian, menampilkan ukuran antrian menggunakan **countQueue()**, lalu menghapus item depan dengan **dequeueAntrian()** dan menampilkan antrian lagi dan menghapus antrian dan mencetak isi dan ukuran antrian

```
Data antrian teller:
1. Shilfi
2. Kelvin
3. (kosong)
4. (kosong)
5. (kosong)
Jumlah antrian = 1
Data antrian teller:
1. Kelvin
2. (kosong)
3. (kosong)
4. (kosong)
5. (kosong)
Jumlah antrian = 1
Data antrian teller:
1. (kosong)
2. (kosong)
3. (kosong)
4. (kosong)
5. (kosong)
jumlah antrian = 0

Process returned 0 (0x0)
Press any key to continue.
```

D. UNGUIDED

1. Ubahlah penerapan konsep queue pada bagian guided dari array menjadi linked list
2. Dari nomor 1 buatlah konsep antri dengan atribut Nama mahasiswa dan NIM Mahasiswa
3. Modifikasi program pada soal 1 sehingga mahasiswa dapat diprioritaskan berdasarkan NIM (NIM yang lebih kecil didahulukan pada saat output).
Noted : Untuk data mahasiswa dan nim dimasukan oleh user

Code:

```
#include <iostream>
#include <string>

using namespace std;

// Node structure for the linked list
struct Node {
    string nama;
    int nim;
    Node* next;
};

Node* front = nullptr; // Front of the queue
Node* back = nullptr; // Back of the queue

// Check if the queue is empty
bool isEmpty() {
    return front == nullptr;
}

// Enqueue function with priority based on NIM
void enqueueAntrian(string nama, int nim) {
    Node* newNode = new Node(nama, nim, nullptr);

    if (isEmpty()) {
        // First element in the queue
        front = back = newNode;
    } else {
        // Insert node in order based on NIM
        if (nim < front->nim) {
            // Insert new node at the front if it has the smallest NIM
            newNode->next = front;
            front = newNode;
        } else {
            // Traverse to find correct position based on NIM
            Node* current = front;
            while (current->next != nullptr && current->next->nim < nim) {
                current = current->next;
            }
            newNode->next = current->next;
            current->next = newNode;

            // Update back pointer if we insert at the end
            if (newNode->next == nullptr) {
                back = newNode;
            }
        }
    }
}

// Dequeue function (removes the front element)
void dequeueAntrian() {
    if (isEmpty()) {
        cout << "Antrian kosong" << endl;
    } else {
        Node* temp = front;
        front = front->next;
        delete temp;

        // If the queue becomes empty, reset back to null
        if (front == nullptr) {
            back = nullptr;
        }
    }
}

// Count elements in the queue
int countQueue() {
    int count = 0;
    Node* current = front;
    while (current != nullptr) {
        count++;
        current = current->next;
    }
    return count;
}
```

```

// Clear the queue
void clearQueue() {
    while (!isEmpty()) {
        dequeueAntrian();
    }
}

// Display all elements in the queue
void viewQueue() {
    if (isEmpty()) {
        cout << "Antrian kosong" << endl;
    } else {
        cout << "Data antrian teller:" << endl;
        Node* current = front;
        int i = 1;
        while (current != nullptr) {
            cout << i << ". Nama: " << current->nama << ", NIM: " << current->nim << endl;
            current = current->next;
            i++;
        }
    }
}

int main() {
    int n, nim;
    string nama;

    cout << "Masukkan jumlah mahasiswa: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        cout << "Masukkan Nama Mahasiswa: ";
        cin >> nama;
        cout << "Masukkan NIM Mahasiswa: ";
        cin >> nim;
        enqueueAntrian(nama, nim);
    }

    viewQueue();
    cout << "Jumlah antrian = " << countQueue() << endl;

    // Example of dequeuing and viewing the queue again
    cout << "Menghapus antrian terdepan..." << endl;
    dequeueAntrian();
    viewQueue();
    cout << "Jumlah antrian = " << countQueue() << endl;

    cout << "Mengosongkan antrian..." << endl;
    clearQueue();
    viewQueue();
    cout << "Jumlah antrian = " << countQueue() << endl;

    return 0;
}

```

- Terdapat struktur node yang mewakili setiap siswa dalam antrian (nama,nim). Front dan back mewakili pointer depan dan belakang antrian yang keduanya ditetapkan ke nullptr agar menunjukkan antrian kosong.
- Terdapat fungsi **isEmpty** berfungsi memeriksa antrian kosong dengan mengembalikan true if front is nullptr. Ada fungsi **enqueueAntrian** yaitu untuk menambahkan siswa baru ke dalam antrian, mengurutkan siswa berdasarkan NIM. Fungsi **dequeueAntrian** untuk menghapus elemen depan antrian, Jika antrian kosong maka akan ditampilkan “Antrian kosong”. Fungsi **countQueue** untuk menghitung dan mengembalikan jumlah node dalam antrian. Fungsi **clearQueue** untuk menghapus semua elemen dalam antrian dengan memanggil berulang kali dequeueAntrianhingga antrian kosong. Dan yang terakhir terdapat fungsi **viewQueue** untuk menampilkan semua siswa dalam antrian sesuai urutan penyimpanannya.

Output:

Program meminta pengguna agar memasukkan jumlah siswa yang akan ditambah dalam antrian. Lalu masukkan nama dan nim dalam antrian. Maka output menampilkan antrian awal dalam urutan dari nomor paling kecil ke besar, Setelah itu siswa dengan nim terkecil dikeluarkan sehingga menyisakan 2 siswa dalam antrian, Lalu semua siswa dikeluarkan dan tersisa antrian kosong.

```
Masukkan jumlah mahasiswa: 3
Masukkan Nama Mahasiswa: shilfi
Masukkan NIM Mahasiswa: 1510
Masukkan Nama Mahasiswa: kelvin
Masukkan NIM Mahasiswa: 1012
Masukkan Nama Mahasiswa: nadya
Masukkan NIM Mahasiswa: 0911
Data antrian teller:
1. Nama: nadya, NIM: 911
2. Nama: kelvin, NIM: 1012
3. Nama: shilfi, NIM: 1510
Jumlah antrian = 3
Menghapus antrian terdepan...
Data antrian teller:
1. Nama: kelvin, NIM: 1012
2. Nama: shilfi, NIM: 1510
Jumlah antrian = 2
Mengosongkan antrian...
Antrian kosong
Jumlah antrian = 0
```